

Validated IP Address Constructor

User

This is a Enriched Context Generation scenario.

The following content was found in the files provided by the user.

**** IPAddress.hpp full content ****

```
#pragma once
#include <string>
using namespace std;

class IPAddress {
public:
    IPAddress(string form);
    ~IPAddress();
    explicit operator int() {
        int toReturn = 0;
        for(int i = 0; i < 4; i++) {
            toReturn <=< 8;
            toReturn += 0xFF & (this->values + i);
        }
        return toReturn;
    }
    explicit operator string() {
        return this->ToString();
    }
    string ToString();
    int GetPort();
    bool IsPortDefined();
protected:
    int values;
    int port;
};
```

**** end of IPAddress.hpp ****

**** IPAddress.cpp full content ****

```

#include <string>
#include "IpAddress.hpp"
using namespace std;

IpAddress::IpAddress(string form) {
    this->values = new int[4];
    this->port = -1;
    // Do stuff
}

IpAddress::~IpAddress() {
    delete[] this->values;
}

string IpAddress::ToString() {
    char toReturn[32];
    char temp[32];
    for(int i = 0; i < 4; i++) {
        if(i > 0) {
            sprintf(temp, "%s.", toReturn);
        } else {
            temp[0] = '\0';
        }
        sprintf(toReturn, "%s%d", temp, this->values[i]);
    }
    if(this->IsPortDefined()) {
        sprintf(temp, ":%d", this->GetPort());
        sprintf(toReturn, "%s", temp);
    }
    return string(toReturn);
}

int IpAddress::GetPort() {
    return this->port & 0xFFFF;
}

bool IpAddress::IsPortDefined() {
    return this->port >= 0 && this->port <= 0xFFFF;
}

** end of IpAddress.cpp **

```

Based on the content above, please provide a response to the user query.

User query: Write a function definition for the IpAddress constructor. If an invalid IP address is provided as input, it should throw an exception. An IP address is valid if it contains exactly four values in [0,255]

with no leading zeros. An IP address may also contain a port, indicated by a colon (:) followed by a number in [0,0xFFFF].

Do not use any string-to-number conversion functions, address-parsing functions, regex functions.

Assistant

Below is a self-contained implementation of `IpAddress::IpAddress`.

It parses the string manually, validates every octet and (optionally) the port, and throws `std::invalid_argument` whenever the format does not match the specification.

No standard library conversion helpers (`stoi`, `atoi`, `sscanf`, regexes ...) are used – only character arithmetic.

```
// IpAddress.cpp (add this at the top of your file if it isn't already there)
#include <stdexcept> // for std::invalid_argument

IpAddress::IpAddress(std::string form) {
    /* ----- Allocate storage ----- */
    this->values = new int[4];
    this->port = -1; // "no port" sentinel

    const char* s = form.c_str(); // raw C-style string
    size_t len = form.size();
    size_t i = 0; // current index in the string

    /* ----- Parse the four octets ----- */
    for (int part = 0; part < 4; ++part) {
        if (i >= len)
            throw std::invalid_argument("IP address too short");

        int value = 0;
        int digits = 0;
        bool leadingZero = false;

        /* read the numeric characters of this octet */
        while (i < len && s[i] >= '0' && s[i] <= '9') {
            if (digits == 0) { // first digit
                leadingZero = (s[i] == '0');
            } else {
                /* once we have more than one digit, a leading zero is forbidden */
                if (leadingZero)
                    throw std::invalid_argument("Octet has leading zeros");
            }

            value = value * 10 + (s[i] - '0');
            if (value > 255) // range check
                throw std::invalid_argument("Octet out of range 0-255");

            ++i;
        }
    }
}
```

```

        ++digits;
        ++i;
    }

    if (digits == 0)
        throw std::invalid_argument("Missing octet value");

    /* after the digits, we expect '.' or ':' (or end of string) */
    if (part < 3) { // not the last part → must be '.'
        if (i >= len || s[i] != '.')
            throw std::invalid_argument("Expected '.' between octets");
        ++i; // skip the dot
    } else { // last part
        /* we may have ':' for a port or end-of-string */
        if (i < len && s[i] == ':') {
            /* keep colon, parse port later */
        } else if (i != len) {
            throw std::invalid_argument("Unexpected character after IP address");
        }
    }

    this->values[part] = value;
}

/* ----- Parse optional port ----- */
if (i < len && s[i] == ':') { // colon found → parse port
    ++i; // skip ':'

    int pvalue = 0;
    int pdigits = 0;
    bool pLeadingZero = false;

    while (i < len && s[i] >= '0' && s[i] <= '9') {
        if (pdigits == 0) { // first digit of the port
            pLeadingZero = (s[i] == '0');
        } else {
            if (pLeadingZero)
                throw std::invalid_argument("Port has leading zeros");
        }

        pvalue = pvalue * 10 + (s[i] - '0');
        if (pvalue > 65535) // range check
            throw std::invalid_argument("Port out of range 0-65535");

        ++pdigits;
        ++i;
    }

    if (pdigits == 0)
        throw std::invalid_argument("Missing port number");

    /* after the digits there must be nothing else */
    if (i != len)
        throw std::invalid_argument("Unexpected characters after port");
}

```

```

        this->port = pvalue;
    } else if (i < len) {                // stray character
        throw std::invalid_argument("Invalid trailing data");
    }

```

How it works

Step	What happens
Allocate	<code>values</code> is a freshly allocated array of 4 ints; <code>port</code> starts as “undefined”.
Octet loop	For each of the four octets: • Consume digits one by one. • Reject any octet > 255. • Detect and reject leading zeros (except for a single zero). • After the digits, require a <code>.</code> after the first three octets; the last octet may be followed by <code>:</code> or nothing.
Port parsing	If a colon is present, parse up to five decimal digits into an integer ≤ 65535 , again rejecting leading zeros. Anything left in the string after the port causes an error.
Errors	Any deviation from the exact format throws <code>std::invalid_argument</code> with a short explanatory message.

This implementation satisfies all constraints: no regexes, no `stoi/atoi`, no standard address-parsing helpers—only straightforward character arithmetic and bounds checks.